FINM 32950: Intro to HPC in Finance

Lecture 2

Chanaka Liyanarachchi

April 1, 2026

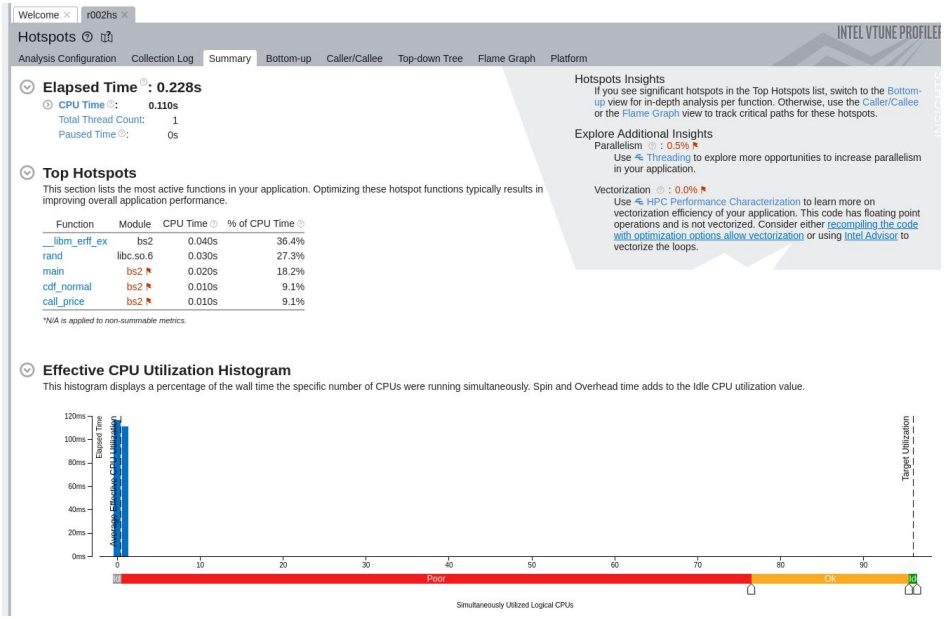
Using Multicore Nodes

Structured Parallel Programming

High-Level Constructs for Parallel Programming

Using Multicore Nodes

- Let's go back to the report from vtune last time and focus on the *Effective CPU Utilization Histogram* section.



- We have over 90 CPU cores but we just use one.
- How do we use more than one CPU cores?

Parallel Programming - Introduction

Parallel Programming

- One natural way to improve performance is to perform multiple tasks simultaneously (in parallel):
  - This idea has been around for a long time.
  - It has become increasingly important in modern computing.
- We have already seen vectorization, which provides parallelism within a single CPU core (instruction-level parallelism).
- Our next focus is multithreading, which allows us to utilize multiple CPU cores for parallel execution.

Learning Parallel Programming

- Modern computers are multi-core systems. `http://www.gotw.ca/publications/concurrency-ddj.htm`
- We should understand how to utilize multicore architectures effectively.
- Otherwise, a significant portion of available computing power remains unused.
- Writing efficient sequential code alone is no longer sufficient.
- Parallel computing is no longer a highly specialized or niche topic; it is a fundamental skill every programmer should develop.

Parallel Computing: Example 1

- Suppose we want to add two arrays (vectors):

$$\begin{bmatrix} a_0 \\ a_1 \\ a_2 \end{bmatrix} + \begin{bmatrix} b_0 \\ b_1 \\ b_2 \end{bmatrix} = \begin{bmatrix} c_0 \\ c_1 \\ c_2 \end{bmatrix}$$

- We can write a simple for loop:

```
for (int i=0; i<N; ++i)
    c[i] = a[i] + b[i];
```

- This loop adds two values and assign it to $c[i]$ for each i in N.
- Usually when we think about programs (thus far), we think about doing things sequentially:
  - Use one data item from each array
  - Executing one instruction at a time
- Each addition is independent of the others (i.e. `c[0]` doesn't depend on `c[1]` and so on) – we can compute them in parallel.

Parallel Computing: Example 2

- Matrix multiplication is another example:

$$A_{I,K}B_{K,J} = C_{I,J} \qquad c_{i,j} = \sum_{k=0}^{K-1} a_{i,k}b_{k,j} \text{ where } 0 \le i < I \text{ and } 0 \le j < J$$

$$\begin{pmatrix} a_{0,0} & a_{0,1} & \cdots & a_{0,k-1} \\ a_{1,0} & a_{1,1} & \cdots & a_{1,k-1} \\ \vdots & \vdots & \ddots & \vdots \\ a_{i-1,0} & a_{i-1,1} & \cdots & a_{i-1,k-1} \end{pmatrix} \begin{pmatrix} b_{0,0} & b_{0,1} & \cdots & b_{0,j-1} \\ b_{1,0} & b_{1,1} & \cdots & b_{1,j-1} \\ \vdots & \vdots & \ddots & \vdots \\ b_{k-1,1} & b_{k-1,2} & \cdots & b_{k-1,j-1} \end{pmatrix}$$

$$= \begin{pmatrix} c_{0,0} & c_{0,1} & \cdots & c_{0,j-1} \\ c_{1,0} & c_{1,1} & \cdots & c_{1,j-1} \\ \vdots & \vdots & \ddots & \vdots \\ c_{i-1,0} & c_{i-1,1} & \cdots & c_{i-1,j-1} \end{pmatrix}$$

- Each element $c_{i,j}$ is independent of others – we can compute them in parallel.

Natural Parallelism

- Many real-world applications exhibit natural parallelism.
- Examples include vector addition and matrix multiplication.
- Our goal in this course is to identify and exploit this natural parallelism to improve performance.
- We will explore three main technologies:
  1. Vectorization ✓
  2. Multicore
  3. GPGPU

Parallel Programming Using Threads

Threads

- To write parallel programs utilizing multiple CPU cores, we break a program into smaller independent *tasks* that can run in parallel.
- These tasks can be executed concurrently using *threads*.
- Each thread represents a separate sequence of execution that can run on a different core.
- Our next focus is to understand how to use threads effectively to perform work in parallel.
- Before that, we will discuss some important concepts and challenges associated with using threads.

Using Threads: Example 1

- Let's extend our familiar *Hello World* example to use a thread to perform a task.
- Task in this case is to write the greeting message to console.

```
#include <iostream>
#include <thread>

void greeting()
{
   std::cout << "Hello, World" << std::endl;
}

int main()
{
   std::thread t(greeting);

   t.join();
}
```

To build, on midway:

```
icc helloworld.cpp -o helloworld -lpthread
```

Using Threads: Hello World Example

- To create a thread, we use `std::thread`.
- The `std::thread` constructor takes a function as an argument.
- Thread execution begins in this function.
- Once the new thread is started, we have two threads in the program:
  1. main thread
  2. new thread (t)
- We use `join()` to make sure the calling thread (main) waits until new thread (t) finishes its execution.
- Thread is defined in `<thread>`
  `http://www.cplusplus.com/reference/thread/thread/`

Thread Join and Detach

- Once a thread is created:
  - calling thread waits until the new thread completes its execution: use `join()`
  - calling thread continues execution without waiting for the new thread: use `detach()`

How Many Threads

- We can create a large number of threads in a program.
- However, the number of threads that can run truly in parallel is limited by the number of CPU cores (and hardware threads).
- If we create more threads than available cores, they will not all run simultaneously.
- Instead, the operating system schedules threads, switching between them using a scheduling algorithm.

Detour: Lambdas (C++)

Lambdas

- We saw the `std::thread` constructor takes a function as an arugment.
- There are other ways to pass an initial function to a thread:
  - using a lambda
  - using a function object

Lambda: Syntax

- The example below shows a very simple lambda which writes a message to console:

```
[] (string s)
{
  cout << s << endl;
}
```

- `[]` is called the capture clause; a lambda is always introduced by the capture clause.
- `()` parenthesis allows us to pass optional arguments to a lambda.
- `{}` lambda body is enclosed by the `{}` brackets.

Creating a Thread

- We can pass a lambda to a thread:

```
std::thread t([]()
{
      cout << "Hello, World" << endl;
});

t.join();
```

Lambda Syntax: Captures

- Lambdas allow us to *capture* parameters by value, or by reference.
- You can use *everything* notation to capture. In this case the compiler captures what's needed (used inside the body) by the lambda.
- `[=]` captures everything by value – allows reads but no write access.
- `[&]` captures everything by reference – allows read and write access.
- `[=,&x]` captures everything by value except `x`; `x` is captured by reference.
- `[&, x]` captures everything by reference except `x`; `x` is captured by value.

Using Threads: Matrix Multiplication

- Matrix multiplication:

$$A_{I,K}B_{K,J} = C_{I,J} \qquad c_{i,j} = \sum_{k=0}^{K-1} a_{i,k}b_{k,j} \text{ where } 0 \leq i < I \text{ and } 0 \leq j < J$$

$$\begin{pmatrix} a_{0,0} & a_{0,1} & \cdots & a_{0,k-1} \\ a_{1,0} & a_{1,1} & \cdots & a_{1,k-1} \\ \vdots & \vdots & \ddots & \vdots \\ a_{i-1,0} & a_{i-1,1} & \cdots & a_{i-1,k-1} \end{pmatrix} \begin{pmatrix} b_{0,0} & b_{0,1} & \cdots & b_{0,j-1} \\ b_{1,0} & b_{1,1} & \cdots & b_{1,j-1} \\ \vdots & \vdots & \ddots & \vdots \\ b_{k-1,1} & b_{k-1,2} & \cdots & b_{k-1,j-1} \end{pmatrix}$$

$$= \begin{pmatrix} c_{0,0} & c_{0,1} & \cdots & c_{0,j-1} \\ c_{1,0} & c_{1,1} & \cdots & c_{1,j-1} \\ \vdots & \vdots & \ddots & \vdots \\ c_{i-1,0} & c_{i-1,1} & \cdots & c_{i-1,j-1} \end{pmatrix}$$

- We can write a serial program:

```
void matrix_multiply(const matrix& m1, const matrix& m2,
   matrix& m3, int I, int J, int K)
{
   for (int i = 0; i < I; ++i)
   {
      for (int j = 0; j < J; ++j)
      {
            m3[i][j] = 0;
            for (int k = 0; k < K; ++k)
            {
                m3[i][j] += m1[i][k]*m2[k][j];
            }
         }
    }
}
```

- Inputs (m1 and m2) are passed by const reference.
- Output (m3) passed by reference.
- Multiplying IxK and KxJ matrices.

- This is a naturally parallel program - each $c_{i,j}$ can be computed in parallel.
- We can parallelize it in many different ways:
  - one thread to calculate each element $c_{i,j}$ – we need numRow * numColumn number of threads
  - one thread for one row – we need numRow number of threads
  - one thread for one column – we need numColumn number of threads
- Which choice is better?

- Suppose we use one thread for each element:

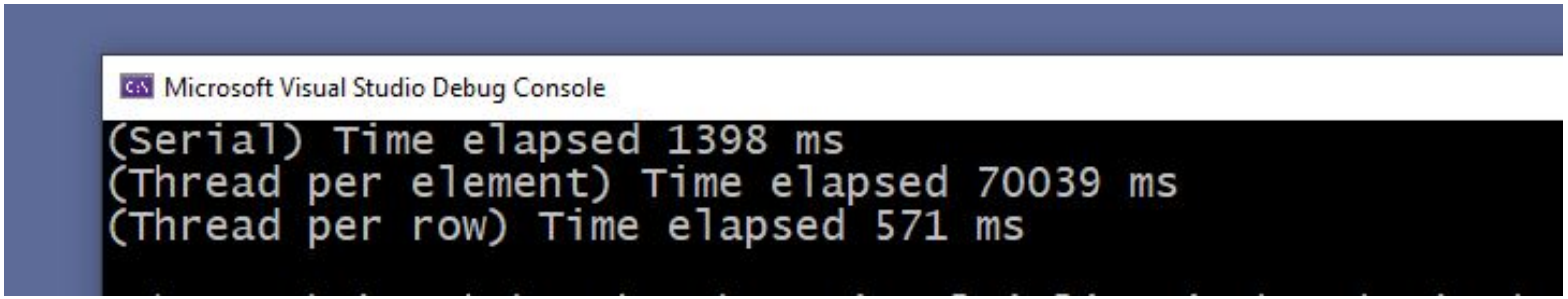
```
void multiply_matrix(const matrix& m1, const matrix& m2,
    matrix& m3, int I, int J, int K)
{
    vector<thread> threads(I*J);

    for (int i = 0; i < I; ++i)
    {
        for (int j = 0; j < J; ++j)
        {
            int idx = i*columns + j;
            threads[idx] = thread([=,&m3]()
            {
                m3[i][j] = 0;
                for (int k = 0; k < K; ++k)
                {
                    m3[i][j] += m1[i][k] * m2[k][j];
                }
            });
        }
    }

    for (thread& t : threads)
    {
        t.join();
    }

}
```

- Or, we can to use one thread for one row:

```
void multiply_matrix(const matrix& m1, const matrix& m2,
    matrix& m3, int I, int J, int K)
{
    vector<thread> threads(I);

    for (int i = 0; i < I; ++i)
    {
        threads[i] = thread([=,&m3]()
        {
            for (int j = 0; j < J; ++j)
            {
                m3[i][j] = 0;
                for (int k = 0; k < K; ++k)
                {
                    m3[i][j] += m1[i][k] * m2[k][j];
                }
            }
       });
     }

    for (thread& t : threads)
    {
        t.join();
    }

}
```

- Note: This program does not use any special features of Intel compiler (i.e. you may use Visual C++ or any other compiler).
- We have 3 implementations now:
  1. sequential work – one thread (i.e. main thread) computes all elements
  2. one thread computes one element
  3. one thread computes one row
- Which program/programs will run faster?

- Let's run these programs and measure time.
- Results depend on the system and the matrix sizes etc.
- Shown below is what I saw for one case (see demo for code) on my laptop:



Observations

- Threads allow us to improve performance by executing work in parallel.
- However, more threads does not necessarily mean better performance. Why?
- If we create more threads than can run in parallel, we introduce overhead:
  - Thread creation and management costs
  - Context switching between threads
- As a result, excessive threading can reduce performance instead of improving it.
- When using threads, we must carefully balance parallelism and overhead.

Sharing Data

- Threads within a program share the same memory space.
- Multiple threads can safely read the same data at the same time (e.g., matrix multiplication).
- What happens if more than one thread writes to the same data simultaneously?
- What if one thread writes while another reads the same data?
- To answer these questions, let's look at an example.

Using Threads: Sharing Data Example

- Let's look at the code snippet below:

```
unsigned long counter = 0;

int numThreads = 10;
vector<thread> threads(numThreads);

for (int j = 0; j < numThreads; ++j)
{
   threads[j] = thread([&counter]()
   {
      for (int i = 0; i<100000; ++i)
      {
         counter++;
      }
   });
}

for (auto& t : threads) t.join();

cout << counter;
```

- To build and run (on Midway):
  1. Get a compute node with 8 CPU cores
     ```
     sinteractive --time=0:30:0 --cpus-per-task=8 --account=finm32950
     ```
  2. Build:
     ```
     icc counter.cpp  -o counter  -lpthread
     ```
  3. Run:
     ```
     ./counter
     ```
- What's the value of `counter`?

Counter Example: Possible Scenario 1

- Suppose we have two threads t1 and t2 running in parallel:
  - t1 reads the value of counter, counter is 0
  - t2 reads the value of counter, counter is 0
  - t1 increments it, counter is 1
  - t2 increments it, counter is 1
  - after 2 increments, the value of counter is 1
- When we use threads, we should expect threads to interleave read and modify operations as shown below:

```
      t1                          t2
   ----------                  --------
|   registerA = counter
|                              registerB = counter
|   counter = registerA + 1
|                              counter = registerB + 1
|
|
v (time/clock-cycle)
```

- In this case, the counter will be incremented only once after two add operations.

Counter Example: Possible Scenario 2

- Suppose we have two threads t1 and t2 running in parallel:
  - t1 reads the value of counter, counter is 0
  - t1 increments it, counter is 1
  - t2 reads the value of counter, counter is 1
  - t2 increments it, counter is 2
  - after 2 increments the value of counter is 2

```
t1                              t2
----                           ----
registerA = counter
counter = registerA + 1
                               registerB = counter
                               counter = registerB + 1
```

- In this case, the counter will be incremented twice (correctly) after two add operations.

Race Conditions

- If two (or more) threads write to the same shared data, the result depends on the order in which the threads ran.
- This problem (bug) is known as a *race condition*.
- How do we avoid race conditions?

Critical Region

- In our example, the counter is a shared variable.
- If multiple threads increment it simultaneously, a race condition occurs.
- The portion of code that accesses shared data is called the *critical section* (or critical region).

```
for (int j = 0; j < numThreads; ++j)
{
   threads[j] = thread([&counter]()
   {
      for (int i = 0; i<100000; ++i)
      {
         counter++;
      }
   });
}
```

- Only one thread should execute the critical section at a time.
- This requirement is known as *mutual exclusion*.

Using a Lock

- One way to achieve mutual exclusion is to use a lock to protect shared data.
- In C++, we use a *mutex* (*mut*ual *ex*clusion).
- A mutex ensures that only one thread can access the protected data at a time.
- To use a mutex:
  - Lock the mutex before accessing shared data
  - Unlock the mutex after accessing shared data
- Only one thread can hold the mutex at any given time.
- Other threads must wait until the mutex is released.
- `http://en.cppreference.com/w/cpp/thread/mutex`

- We can re-write the counter example using a mutex:

```
for (int j = 0; j < numThreads; ++j)
{
   threads[j] = thread([&]()
   {
       for (int i = 0; i<100000; ++i)
       {
         count_mutex.lock();
         counter++;
         count_mutex.unlock();
       }
   });
}
```

- The critical section is now protected using a mutex.
- This ensures that only one thread updates the counter at a time.
- Race conditions are avoided.

Deadlocks

- What happens if we lock a mutex and forget to unlock it?
- No thread will be able to access the shared resource again.
- This situation is known as a *deadlock*.
- Deadlocks can also happen due to exceptions:

```
mutex.lock();

critical_region; // can throw an exception

mutex.unlock();
```

- If an exception occurs before unlocking, the mutex remains locked.
- Proper handling is needed to prevent deadlocks.

std::lock_guard

- The `std::lock_guard` class provides an RAII (Resource Acquisition Is Initialization) technique for mutexes:
  - Locks the mutex when the `lock_guard` object is created
  - Automatically unlocks the mutex when the object goes out of scope
- Advantages of `std::lock_guard`:
  - Prevents errors from forgetting to unlock
  - Provides exception safety
- Defined in `<mutex>`

- The code snippet below shows how to lock a piece of shared data using a `std::mutex` using this technique.

```
unsigned long count = 0;
std::mutex count_mutex;

int numThreads = 10;
vector<thread> threads(numThreads);

for (int j = 0; j < numThreads; ++j)
{
   threads[j] = thread([&]()
   {
       for (int i = 0; i<100000; ++i)
       {
         std::lock_guard<std::mutex> guard(count_mutex);
         count++;
      }
   });
}
```

Structured Parallel Programming

Road Map

We have discussed several key concepts in multithreading:

- Creating and managing threads
- Thread synchronization
- Critical sections
- Race conditions
- Mutual exclusion
- Locks and mutexes
- Deadlocks
- Atomics

These concepts form the foundation for writing safe and efficient parallel programs.

Problem: Low CPU Utilization

- Our goal: leverage threads to write parallel programs that fully utilize CPU resources.
- Suppose we profile an application and notice low CPU utilization:

Basic Hotspots Hotspots by CPU Usage viewpoint (change)

Analysis Target | Analysis Type | Collection Log | Summary | Bottom-up | Caller/Callee | Top-down Tree | Pla

Elapsed Time: 1.575s

CPU Time:	1.003s
Total Thread Count:	1
Paused Time:	0s

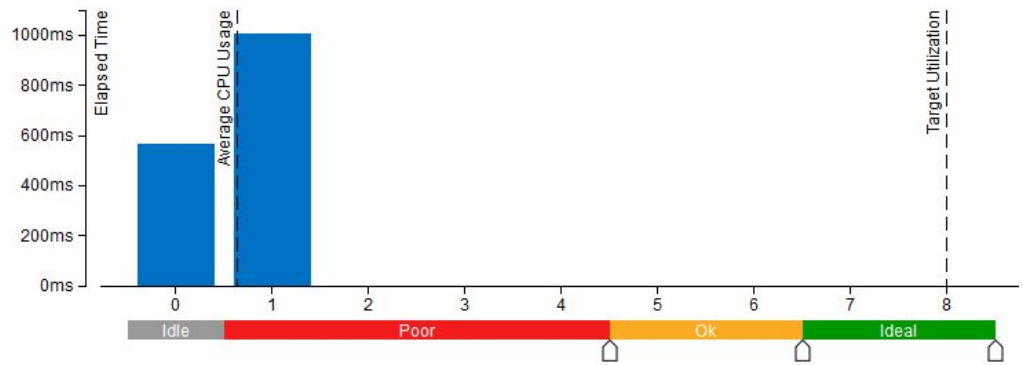
Top Hotspots

This section lists the most active functions in your application. Optimizing these hotspot functions typically results in improving ov

Function	Module	CPU Time
rand	ucrtbased.dll	0.336s
cdf_normal	BlackScholes.exe	0.120s
logf	BlackScholes.exe	0.106s
__libm_exp_s9	libmmd.dll	0.090s
__libm_log_s9	libmmd.dll	0.080s
[Others]		0.270s

CPU Usage Histogram

This histogram displays a percentage of the wall time the specific number of CPUs were running simultaneously. Spin and Overhe



Pattern-Based Approach

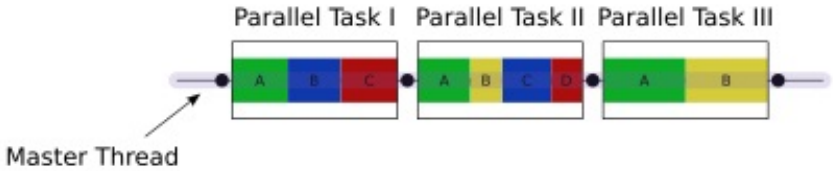
- Designing parallel programs effectively can be challenging.
- We will use a *structured, pattern-based approach* to simplify design and implementation.
- Patterns are widely used in software engineering to capture best practices for solving common problems and use cases.
- *Parallel Patterns* capture known solutions and best practices specifically for parallel program design.
- A program can use one or more parallel patterns to structure its parallelism.

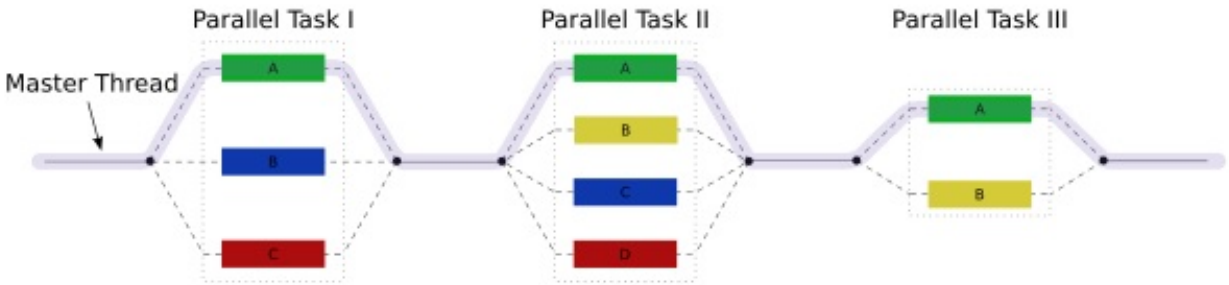
- In this course, we will:
  - Introduce common parallel patterns
  - Implement them in homework assignments
  - Use them as high-level design tools for writing parallel programs
- This provides a structured approach to applying multithreading concepts to real-world problems.

Parallel Patterns

Fork-join

- Very common programming pattern as shown below[1]



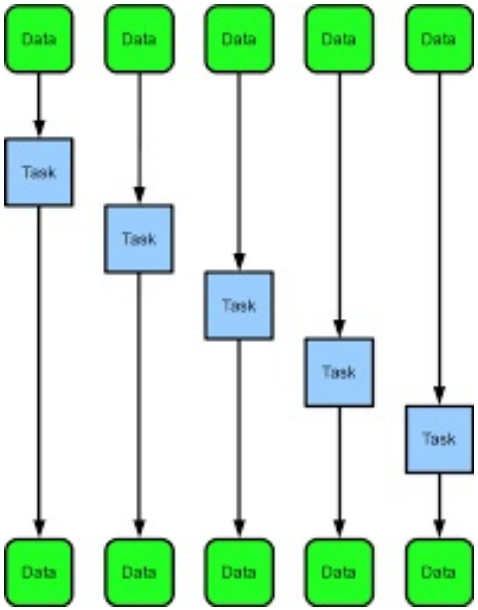


- Generally, this pattern involves:
  - serial work
  - parallel work
  - wait for all parallel work to finish
  - go to next task

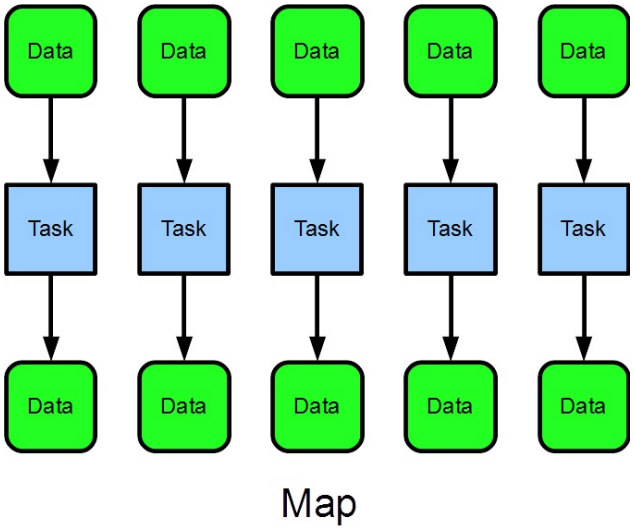
[1] figure source: wikipedia

Map

- Map pattern involves applying an operation to all elements in a sequence.
- This operation produces a new collection with the same shape as the input.
- Serial execution of this pattern takes the form of:



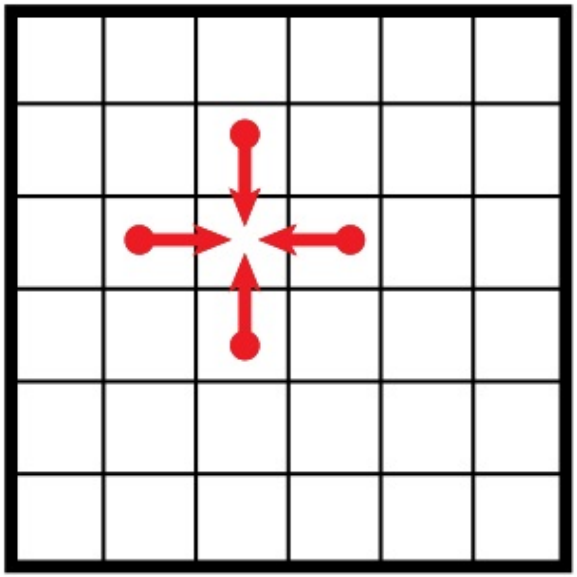
- Parallel execution of this pattern takes the form of:



- The operation is known as *elemental function*.
- The elemental function should not modify shared (global) data that other instances depend on.

Stencil

- Here the elemental function access an element in a sequence and its neighbors.



Reduction

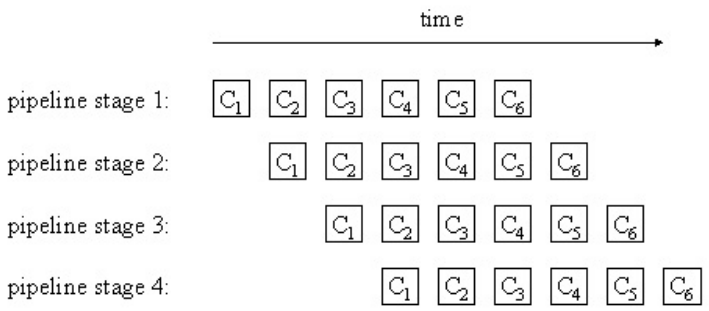
- A reduction combines elements in a collection into a single element.
- A simple example would be computing the sum of elements in a vector.
- A function used in reduction is known as the *combiner function*.

Pipeline

- Pipeline is a very common example/concept in everyday life.
- E.g. Assembly line of a car factory.

- Performs a specific task on an item at each station.
- Move the item to the next station for the next task.

- Next item takes the place of an item when it leaves a station.
- All stations are busy.



- Widely used in Processor/FPGA designs.

Many Patterns

- We looked at some patterns we plan to use in this course.
- There are many more:
  - Scan
  - Scatter
  - Gather
  - Futures
  - Search
  - ...
- A detailed discussion of all common parallel patterns is beyond the scope of this course.
- Recommended reading: *Structured Parallel Programming*[2] is a great resource for further studies on the subject.

[2] See course references for details

Data Parallelism

- Data parallelism is a dominant paradigm in modern high-performance computing
- The same operation is applied independently across many data elements
- Aligns naturally with GPUs and vectorized hardware
- Enables scalable performance with minimal coordination
- Most effective when computations are uniform across large datasets
- Works best when dependencies between data elements are limited

Other forms of parallelism exist:

- Task parallelism: different tasks execute concurrently
- Pipeline parallelism: data flows through staged processing
- ...

Due to time constraints, this course focuses on data parallelism as it provides the most direct and widely applicable path to performance gains.

Data-Parallel Pattern Examples

- Map: apply a function independently to each element
- Reduce: aggregate values into a single result
- Stencil: compute values using local neighborhoods
- Scan: compute prefix operations

Map–Reduce refers here to the general computational pattern of mapping followed by aggregation, not to distributed frameworks such as Hadoop.

In practice, real systems combine these patterns.

Example:

$$\text{Map} \rightarrow \text{Reduce} \rightarrow \text{Scan} \rightarrow \text{Stencil}$$

Assignment 1 - Required

- Due: April 8 by midnight.
- Implement any one of the following patterns using `std::thread` to demonstrate how it works:
  - Fork-Join
  - Map
  - Stencil
  - Reduction
- Your program does not need to perform anything *meaningful*, but it can if you wish.
- For instance, the elemental function can simply output a `cout` statement to illustrate the pattern.
- Think of it as showing a friend how the pattern works through code.

Using Threads: Challenges

- We've seen how to use threads to perform work in parallel.
- Using `std::thread` directly is not easy, since it is a low-level construct:
  - Deciding how many threads to use is not straightforward
  - Achieving load balancing is difficult
  - Solutions may not scale well
- The concepts we've discussed are crucial:
  - Provides a strong foundation for writing parallel programs
  - Helps understand critical sections, data races, locks, and related issues
- Next, we will explore a higher-level approach to using threads more effectively.

High-Level Constructs for Parallel Programming

High-Level Constructs for Concurrency

- High-level programming constructs allow us to express parallelism without managing low-level details.
- Useful for both beginners and experienced programmers.
- Advantages:
  - Clarity and readability
  - Ease of use, improving productivity
  - Less error-prone
  - Scalability

Popular High-Level Constructs

- Common high-level constructs for parallel programming include:
  1. OpenMP – widely used, especially for loops and shared-memory parallelism
  2. Intel Threading Building Blocks (TBB) – rich feature set for task-based parallelism
  3. Microsoft Parallel Patterns Library (PPL) – similar features to TBB
  4. C++ Standard Library – provides concurrency support via threads, async, futures, and parallel algorithms
- You are not limited to a single model—mix-and-match as needed.

OpenMP (**Open** specification for **M**ulti **P**rocessing)

OpenMP Overview

- Popular parallel programming model.
- Simplifies development of parallel code using:
  1. Compiler directives
  2. Library functions
  3. Environment variables
- You don't need to use all three; some features overlap.
- Based on a specification[3] developed by industry (IBM, Intel, Oracle, others).
- Platform- and compiler-neutral approach to shared-memory parallelism.

[3] `www.openmp.org`

OpenMP: Platform & Language Support

- Works on multiple OSs:
  1. Windows
  2. Various flavors of Unix/Linux
- Compiler support:
  1. Visual C++ (OpenMP 2.0) [4]
  2. Intel Compiler, GCC – support newer OpenMP versions (4.x, some 5.x features)
- Supported languages:
  1. C++
  2. C
  3. Fortran

[4] https://devblogs.microsoft.com/cppblog/improved-openmp-support-for-cpp-in-visual-studio/

OpenMP: Hello World Example

- Simple example using compiler directives:

```
#include <omp.h>
#include <iostream>
using namespace std;

int main() {
   #pragma omp parallel
   {
       cout << "Hello, world" << endl;
   }
}
```

- The compiler automatically generates threads for the parallel region.
- Threads share memory; no need to manage threads manually.

OpenMP: Specifying Number of Threads

- Use the optional clause `num_threads(n)`:

```
#pragma omp parallel num_threads(2)
{
    cout << "Hello, world" << endl;
}
```

- Each thread executes all instructions in the parallel block.
- Number of threads can also be controlled via environment variables.

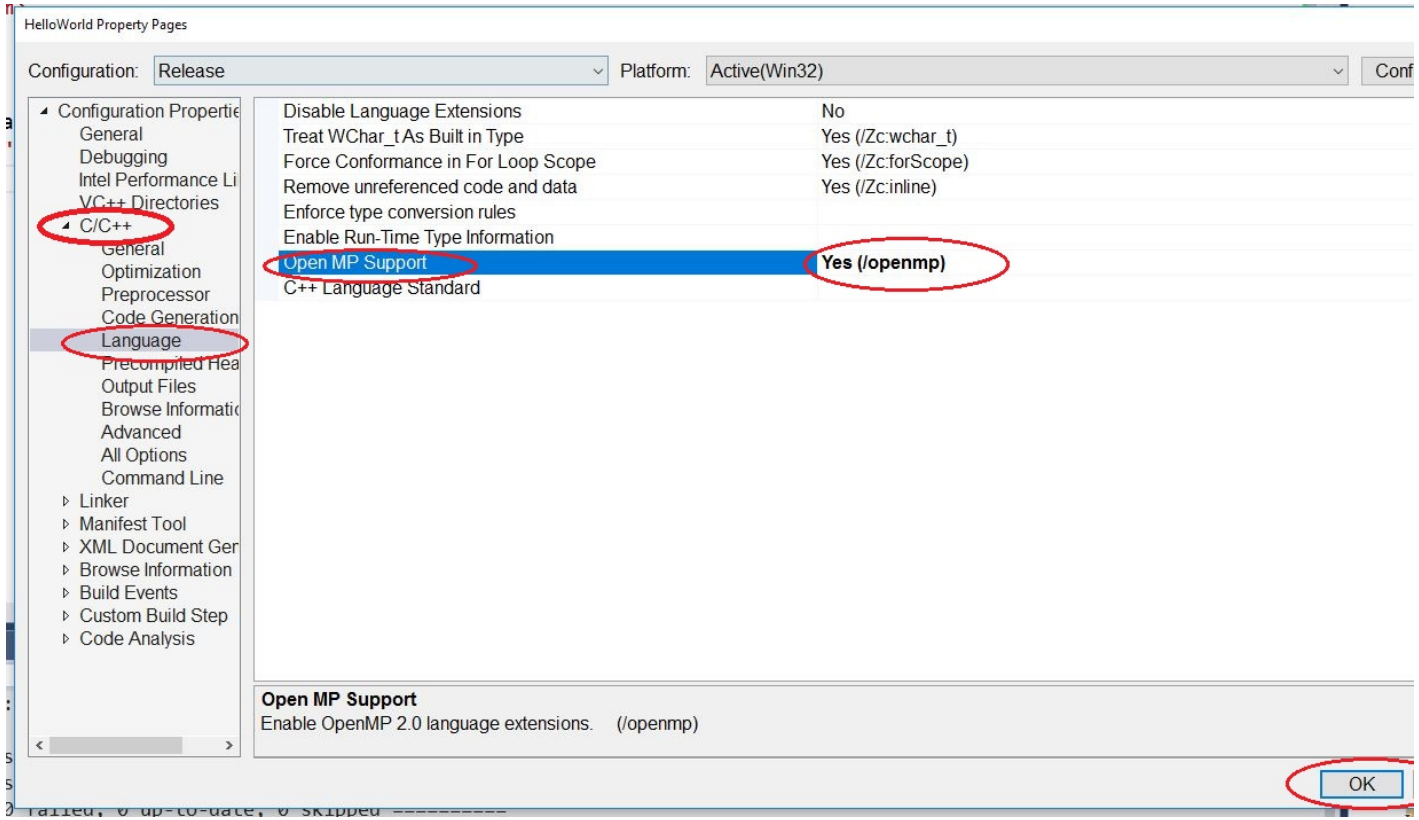
Enabling OpenMP Support

- Intel Compiler (Linux):

  `icc -qopenmp hello.cpp -o hello`

- Visual C++ (Windows):

  `Properties -> C/C++ -> Language -> OpenMP Support -> Ye`



OpenMP Directive Format

```
#pragma omp directive-name [clause]
```

- `#pragma` provides extra information to the compiler.
- `omp` indicates OpenMP.
- `directive-name` is required.
- `[clauses]` are optional; they control specific behavior.

Parallel Loops: #pragma omp parallel for

- Easily parallelize loops:

```
#pragma omp parallel for
for (int i = 0; i < 10; ++i) {
    c[i] = a[i] + b[i];
}
```

- Compiler handles thread creation, assignment, and joining.
- Reduces boilerplate and potential errors from low-level thread management.

OpenMP: Matrix Multiplication Example

- Parallelize by row using `#pragma omp parallel for`:

```
#pragma omp parallel for
for (int i = 0; i < rows; ++i)
   for (int j = 0; j < cols; ++j)
       m3[i][j] = 0;
       for (int k = 0; k < rows; ++k)
           m3[i][j] += m1[i][k] * m2[k][j];
```

- Exercise: Change number of threads; observe performance differences.

OpenMP Sections

- Execute different tasks in parallel:

```
#pragma omp parallel
{
   #pragma omp sections
   {
       #pragma omp section
       task1();

       #pragma omp section
       task2();
   }
}
```

- Each section runs independently; useful for task-level parallelism.

Thread Scheduling and Load Balancing

- OpenMP scheduling policies:
  - static – pre-assigns iterations to threads
  - dynamic – assigns iterations as threads become available
- Static is efficient for uniform workloads; dynamic improves load balancing for uneven workloads.
- Example of dynamic scheduling:

```
#pragma omp parallel for schedule(dynamic) num_threads(4)
for (int i = 0; i < 16; ++i) {
   // simulate work
}
```

Synchronization Constructs

- `barrier` – synchronizes threads at a point.
- `critical` – ensures mutual exclusion (global lock).
- `critical(name)` – named lock for selective serialization.
- `master` – code block executed only by master thread.
- `atomic` – ensures atomic updates to a variable.

```
#pragma omp parallel for
for (int i = 0; i < 100000; ++i) {
    #pragma omp atomic
    counter++;
}
```

Reductions

- Shown below is a simple example for reduction:

```
sum = 0;
for (int i = 0; i < 10; i++)
{
    sum += a[i];
}
```

- `sum` is a shared variable – doing this in parallel is not trivial.
- The OpenMP reduction clause is used to accumulate a value in parallel easily.

```
int sum_sq = 0;

#pragma omp parallel for reduction(+:sum_sq)
for (int i = 0; i < 5; ++i)
{
   sum_sq += i*i;
}
std::cout << sum_sq << std::endl;
```

- Reduction is supported for the following operators:
  `+, -, *, &, |, ^, &&, ||`

Matrix Multiplication: Serial to Parallel

Matrix Multiplication: Serial to Parallel

- We use matrix multiplication to illustrate parallel programming models.
- Serial implementation:

```
void MatrixMultiply(const matrix& m1, const matrix& m2,
                    matrix& m3,
                    int I, int J, int K)
{
    for (int i = 0; i < I; ++i)
    {
        CalculateRow(/* args */);
    }
}
```

Matrix Multiplication: Using std::thread

- Parallel version using low-level threads:

```
void MatrixMultiply(const matrix& m1, const matrix& m2,
                    matrix& m3,
                    int I, int J, int K)
{
    vector<thread> threads(I);

    for (int i = 0; i < rows; ++i)
    {
        threads[i] = thread([&]()
        {
            CalculateRow(/* args */);
        });
    }

    for (thread& t : threads)
    {
        t.join();
    }
}
```

Matrix Multiplication: Using OpenMP

- Parallel version using OpenMP:

```
void MatrixMultiply(const matrix& m1, const matrix& m2,
                    matrix& m3,
                    int I, int J, int K)
{
    #pragma omp parallel for
    for (int i = 0; i < I; ++i)
    {
        CalculateRow(/* args */);
    }
}
```

- Advantages of OpenMP:
  - Clarity
  - Ease of use (increased productivity)
  - Less error prone
  - Scalability

Intel Parallel Programming Models

Introduction

- Intel toolkit offers a wide range of support for high-performance and parallel computing.
- Key features we've seen:
  - Tools: e.g., VTune for profiling, Advisor for vectorization help
  - SVML
- Today we will introduce two Intel toolkit features for parallel Monte Carlo simulations:
  - Threading Building Blocks (TBB)
  - Math Kernel Library (MKL)

Threading Building Blocks (TBB)

TBB Overview

- Intel library for high-level *task-based* parallel programming.
- Supports shared-memory parallelism with many useful features.
- Developer guide: `https://www.intel.com/content/www/us/en/docs/onetbb/developer-guide-api-reference/2021-6/onetbb-developer-guide.html`

Task-Based Programming with TBB

- Use a ‘tbb::task_group‘ to run tasks in parallel:

```
tbb::task_group g;

g.run( [&]() { work1(); } );
g.run( [&]() { work2(); } );
g.run( [&]() { work3(); } );

g.wait();
```

- ‘g.run()‘ schedules a task, ‘g.wait()‘ waits for all tasks to finish.

Detour: Function Objects (C++)

Function Objects

- Function objects are objects that behave like functions.
- Achieved by overloading `operator()` for a class.
- They can have member variables and functions — functions with state.
- Widely used in C++ (e.g., STL, threading, TBB tasks).
- We'll use them with threads and TBB.

Creating a Thread with a Function Object

- Example:

```
class ThreadTask
{
private:
    int threadID_;

public:
    ThreadTask(int id) : threadID_(id) {}

    void operator()() const
    {
        std::cout << "Hello, World, " << threadID_ << std::endl;
    }
};
```

- Usage:

```
std::thread t(ThreadTask(2));
t.join();
```

- Thread executes the overloaded `operator()` of the object.

Task-Based Programming with TBB

- Example using ‘tbb::task_group‘:

```
const int NumTasks = 10;
tbb::task_group group;

for (int i = 0; i < NumTasks; ++i)
{
    group.run(ThreadTask(i));
}

group.wait();
```

Task-Based Programming: Advantages

- Tasks allow high-level logical design, abstracting away threads.
- Number of threads is automatically managed:
  - Too many threads $\rightarrow$ overhead, poor performance
  - Too few threads $\rightarrow$ idle CPU resources
  - Optimal number depends on system
- Using tasks eliminates the need to manually choose thread counts.

Task Scheduler

- TBB task scheduler:
  - Creates the correct number of threads based on system resources
  - Performs load balancing automatically
- Manages threads efficiently:
  - Threads are not created for every task $\rightarrow$ less overhead
  - Efficient scheduling compared to OS threads
- Scheduler uses work-stealing and smarter allocation:
  - Threads: OS fair scheduling $\rightarrow$ may be inefficient
  - Tasks: TBB scheduler starts tasks when they can make progress